

Data Storage Systems
University of Minho

Guide 3
LSM Key-Value Stores: LevelDB

2026

1 Setup

Use the VM from *Guide0* and the FUSE file system you have implemented in *Guide2* for the next exercises. First, we must prepare the software and OS environment for this guide.

1. Install compilation and monitoring tools.

```
sudo apt install cmake
sudo apt install dstat
```

2. Install LevelDB.

```
git clone --recurse-submodules https://github.com/google/leveldb.git
cd leveldb
mkdir build && cd build
cmake -DCMAKE_CXX_STANDARD=17 -DCMAKE_BUILD_TYPE=Release .. && cmake --build .
```

3. Copy the source code provided along with the guide to the VM.

2 LevelDB

The LevelDB library provides a persistent log-structured merge tree key-value store and manages data in a key-value pair format (<https://github.com/google/leveldb>).

1. Update the Makefile with the correct path to the LevelDB library headers and binary file, so you can compile the code.

```
CXXFLAGS := -std=c++17 -I/path/to/leveldb/include
LD_FLAGS := -L/path/to/leveldb/build -lleveldb -static
```

2. Mount the FUSE file system implemented in *Guide 2*.

```
sudo ./passthrough /mnt/fs -omodule="subdir,subdir=/backend" -oallow_other -f
```

3. Explore the `myapp.cpp` file. It provides a very simple application that uses LevelDB as a storage engine. Try to add/update, get, scan, and delete key-value pairs. Ensure the KVS is placed in the path mounted by your FUSE file system (e.g., `/mnt/fs/myapp-kvs`).

3 Exercises

1. Implement the `simple_put`, `simple_get`, `simple_scan`, and `simple_delete` functions to add/update, get, scan, and delete key-value pairs, respectively.
Note: Check the LevelDB documentation (<https://github.com/google/leveldb/blob/main/doc/index.md>)
2. Implement four workloads – `writeseq`, `writerand`, `readseq`, `readrand` – that enable writing/reading to/from the key-value store with sequential and random key distributions. Measure the execution time and throughput of each workload. Consider using different number of workers (`num_workers`) and value sizes (`value_size`).
3. Extend the debugging information from the FUSE file system to collect the number of `fsync` and `unlink` calls invoked by LevelDB and analyze which files are being flushed and removed. Further, expand the current debugging information of the `open` and `create` system calls to analyze which flags are being invoked (i.e., `O_RDONLY`, `O_WRONLY`, ...).
4. Instrument the FUSE file system to identify specific I/O patterns when running the different workloads implemented in exercise 2.
 - How many threads are submitting requests to FUSE?
 - **Write amplification:** is there any difference between the bytes written to the key-value store and the bytes written to the file system?
 - **Read amplification:** is there any difference between the bytes read from the key-value store and the bytes read from the file system?
 - Are there any read operations when executing the write-only workloads? Are there any write operations when executing the read-only workloads?

Note1: Before running read-based workloads, ensure that the key-value store contains the key-value pairs you want to access.

Note2: Before each write-based workload, remove the key-value store directory to ensure each experiment is executed with the same state.

4 Extras / Homework

1. Implement a function that allows the implementation of a set of key-value pairs atomically (similar to a database transaction).
Hint: Explore the `WriteBatch` API from LevelDB (<https://github.com/google/leveldb/blob/main/include/leveldb/db.h#L78>).
2. Due to the inherent design of LSM-based key-value stores, a single `Get()` call may involve multiple reads from disk. Explore different mechanisms from LevelDB to reduce the number of reads from the file system and improve the latency of `Get()` calls.
Hint: Explore the `Cache` (<https://github.com/google/leveldb/blob/main/include/leveldb/cache.h>) and `BloomFilter` (https://github.com/google/leveldb/blob/main/include/leveldb/filter_policy.h) mechanisms from LevelDB.

Learning outcomes

Experiment with the LevelDB key-value store. Implement different workloads and access patterns for managing data in a key-value store. Understand the I/O patterns at the file system level caused by the key-value store.